Jun 23, 2019 • 8 min read

Portable Computer Vision: Tensorflow 2 on a Raspberry Pi

Leigh Johnson

Table of Contents

Tiny, low-cost object detection and classification.

Part 1—Introduction

Terms & References 📚

Part 2—Build List ✅

- Starter Kit
- Coral USB Edge TPU Accelerator (Optional)
- Custom Build

Part 3— Raspberry Pi Setup 🍰

Part 4— Primary Computer: Download & Install Dependencies

Part 5— Primary Computer: create configuration files

- .env/my-inventory.ini
- .env/my-vars.json

Part 6— Raspberry Pi: Install Dependencies

Subscribe

Part 7— Introduction to CNNs (convolutional neural networks)
Modeling the Visual Cortex
Visualizing a Convolution Operation: 2D
Visualizing a Convolution Operation: 3D
Do you want to learn more about CNNS?
Part 8—Deploy Pre-trained Model (MobileNetV2)
Live Demo (using TensorFlow 2.0)
Wrapping Up

Are you just getting started with computer vision, TensorFlow, or Raspberry Pi? Perfect, this post is for you!

Tiny, low-cost object detection and classification.

Part 1—Introduction

For roughly $100 USD, you can add deep learning to an embedded system or your next internet-of-things project.

Are you just getting started with machine/deep learning, TensorFlow, or Raspberry Pi? Perfect, this post is for you!

By the end of this post, you'll know:

1. How to setup Raspberry Pi Camera and install software dependencies.
2. Basics of Convolutional Neural Networks for image classification
3. How to deploy a pre-trained model (**MobileNetV2**) **to** Raspberry Pi
4. Convert a model to **TensorFlow Lite,** a model format optimized for embedded and mobile devices.
5. Accelerate inferences of any **TensorFlow Lite** model with Coral's **USB Edge TPU Accelerator** and **Edge TPU Compiler.**

Terms & References 📚

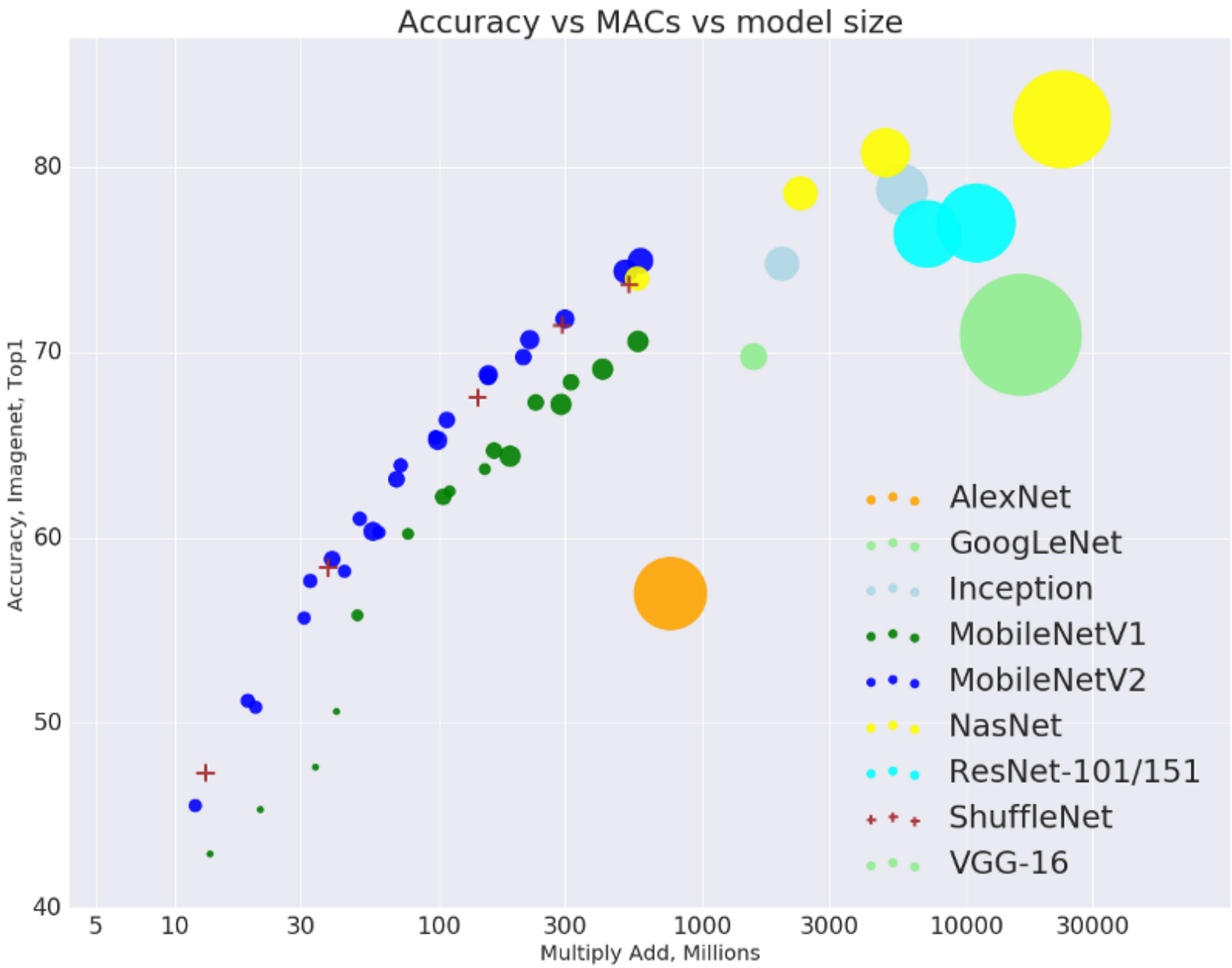
Raspberry Pi—a small, affordable computer popular with educators, hardware hobbyists, and roboticists. 🤖

TensorFlow—an open-source platform for machine learning.

TensorFlow Lite—a lightweight library for deploying **TensorFlow** models on mobile and embedded devices.

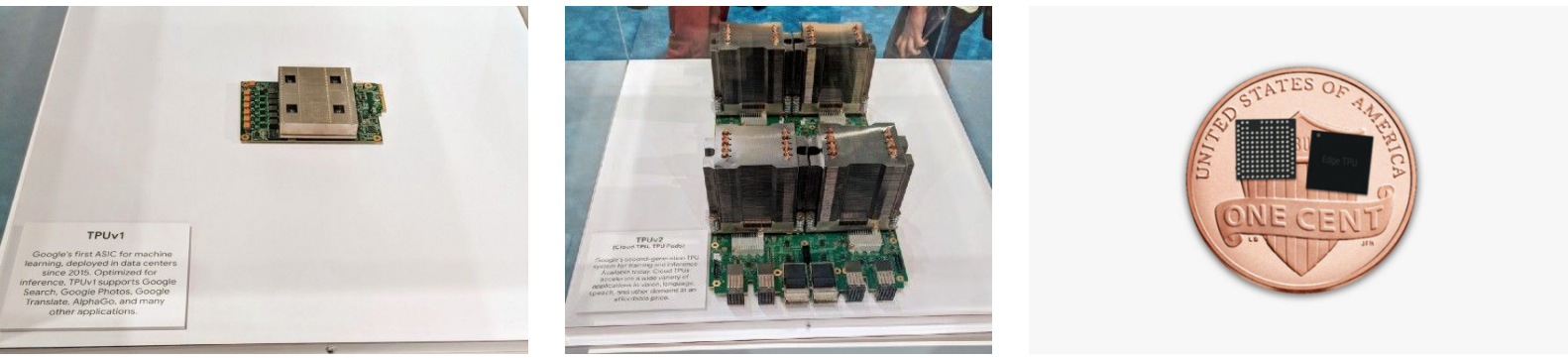
Convolutional Neural Network—a type of deep-learning model well-suited for image classification and object detection applications.

MobileNetV2—a state-of-the-art image recognition model optimized for performance on modest mobile phone processors.



MobileNetV2: The Next Generation of On-Device Computer Vision Networks

Edge TPU—a tensor processing unit (TPU) is an integrated circuit for accelerating computations performed by **TensorFlow.** The **Edge TPU** was developed with a small footprint, for mobile and embedded devices “at the edge”



TPUv1 (left), TPUv2 (middle), Edge TPU (right)

Part 2—Build List ✅

Starter Kit

If you're just getting started with Raspberry Pi, I recommend the Pi Camera Pack ($90) by Arrow. It includes everything you need begin immediately:

- 5V 2.4A MicroUSB Power Supply
- 320×240 2.8" TFT Model PiTFT Resistive Touch-screen
- Raspberry Pi 3 Model B
- Raspberry Pi Camera v2
- Plastic Case
- 8GB MicroSD Card with NOOBS installation manager pre-loaded

Coral USB Edge TPU Accelerator (Optional)

You can compile **TensorFlow Lite** models to run on Coral's USB Accelerator (Link), for quicker model predictions.

Real-time applications benefit significantly from this speed-up. An example would be the decision-making module of an autonomous self-driving robot.

Some applications can tolerate a higher prediction speed and might not require TPU acceleration. For example, you would not need TPU acceleration to build a smart doggie door that unlocks for your pooch (but keeps raccoons out).

If you're just getting started, skip buying this component.

Are you not sure if you need the USB Accelerator? The MobileNet benchmarks below might help you decide. The measurements below depict inference speed (in ms)—lower speeds are better!

Board	MobileNet v1 (ms)	MobileNet v2 (ms)
Coral Dev Board	15.7	20.9
Coral USB Accelerator	49.3	58.1
NVIDIA Jetson Nano (TF)	276.0	309.3
NVIDIA Jetson Nano (TF-TRT)	61.6	72.3
Movidius NCS	115.7	204.5
Intel NCS2	87.2	118.6
MacBook Pro	33.0	71.0
Raspberry Pi (TF Lite)	271.5	379.6
Raspberry Pi (TF)	480.3	654.0

Source: Benchmarking TensorFlow and TensorFlow Lite on the Raspberry Pi by Alasdair Allan

Custom Build

If you already have a Raspberry Pi or some components laying around, the starter kit might include items you don't need.

Here are the parts I used for my own builds (approximately $250 / unit).

- Raspberry Pi Model 3 B+ ($35)
- Raspberry Pi Camera v2 ($30)
- Coral USB Edge TPU Accelerator—accelerates model inferencing ($75, link)
- Pi Foundation Display—7" Touchscreen Display ($80, link)
- SmartiPi Touch Stand ($25, link)
- Adjustable Pi Camera Mount ($5, link)
- Flex cable for RPi Camera 24" ($3, link)

I would love to hear about your own build list! ❤️ Tweet me @grepLeigh or comment below.

Part 3— Raspberry Pi Setup 🍰

If you purchased an SD card pre-loaded with NOOBS, I recommend walking through this overview first: Setting up your Raspberry Pi

Before proceeding, you'll want to:

- Connect your Pi to the internet (doc)
- SSH into your Raspberry Pi (doc)

Part 4— Primary Computer: Download & Install Dependencies

rpi-vision is a set of tools that makes it easier for you to:

- Install a lot of dependencies on your Raspberry Pi (TensorFlow Lite, TFT touch screen drivers, tools for copying PiCamera frame buffer to a TFT touch screen).
- Deploy models to a Raspberry Pi.
- Train new models on your computer or Google Cloud's AI Platform.
- Compile 8-bit quantized models for an Edge TPU.

1. Clone the rpi-vision repo on your **primary computer** (not your Raspberry Pi)

```
$ git clone git@github.com:leigh-johnson/rpi-vision.git && cd rpi-vision
```

2. On your **primary computer**, create a new virtual environment, then install the rpi-vision package.

```
$ pip install virtualenv; virtualenv -p $(which python3) .venv && source .venv/
```

3. Verify you can **SSH into your Raspberry Pi** before proceeding.

If you're using the default Raspbian image, your Pi's hostname will beraspberrypi.local

```
$ ssh pi@raspberry.local
```

Part 5— Primary Computer: create configuration files

rpi-vision uses **Ansible** to manage deployments and tasks on your Raspberry Pi. **Ansible** is a framework for automating the configuration of computers.

Create 2 configuration files required by Ansible:

.env/my-inventory.ini

If you’re using a custom hostname for your Pi, replace raspberrypi.local.

```
tee -a .env/my-inventory.ini <<EOF
[rpi_vision]
raspberrypi.local

[rpi_vision:vars]
ansible_connection=ssh
ansible_user=pi
ansible_python=/usr/bin/python3
EOF
```

.env/my-vars.json

If you’re using a custom hostname for your Pi, replace raspberrypi.local.

```
tee -a .env/my-vars.ini <<EOF
{
  "RPI_HOSTNAME": "raspberrypi.local",
  "VERSION": "release-v1.0.0"
}
EOF
```

Part 6— Raspberry Pi: Install Dependencies

```
$ make rpi-install
```

You'll see the output of an **Ansible playbook**. **Ansible** is a framework for automating the configuration of computers.

A quick summary of what's being installed on your Pi:

- rpi-vision repo
- rpi-fbcp (a tool for copying framebuffer from PiCamera to TFT touch screen display)
- TFT touch screen drivers and X11 configuration

You can inspect the tasks run on your Raspberry Pi by opening playbooks/bootstrap-rpi.yml

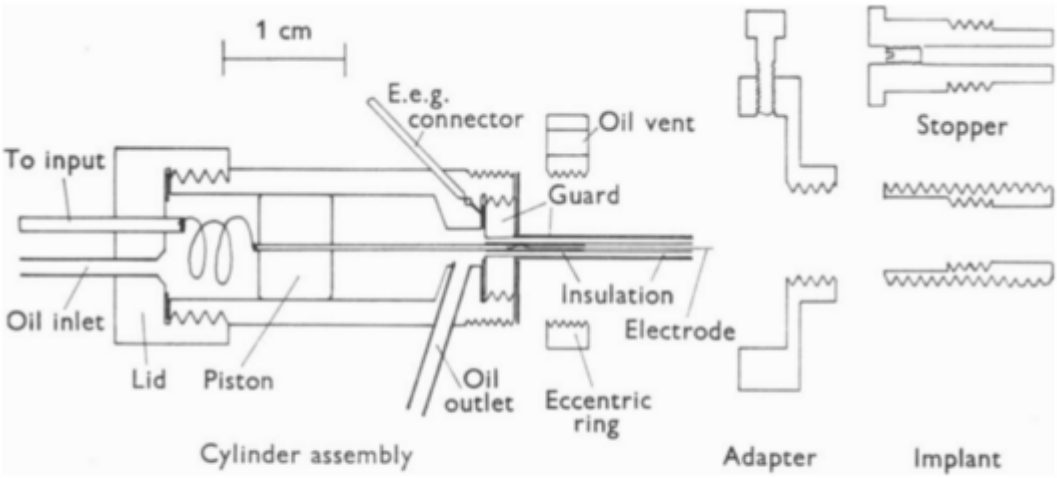
While the installation is running, read through the next section to learn ***how*** CNNS work and ***why*** they are useful for **computer vision** tasks.

Part 7— Introduction to CNNs (convolutional neural networks)

CNNs are the key technology powering self-driving cars and image search engines. The technology is common for computer vision, but can also be applied to any problem with a **hierarchical pattern in the data**, where a **complex pattern** can be **assembled from simpler patterns**.

Modeling the Visual Cortex

In the late 1950s and 1960s, David H. Hubel and Torton Wielson performed experiments on cats and monkeys to better understand the visual cortex.



Text-fig. 1. Diagram of hydraulic micropositioner and implant. Lid, piston and cylinder are made of Perspex; guard is pure silver, insulated inside with nylon sleeve; eccentric ring and adapter are aluminium alloy; implant and stopper are Kel-F. Rubber gaskets are shown between lid and cylinder, and above and below flange of guard.

SINGLE UNIT ACTIVITY IN STRIATE CORTEX OF UNRESTRAINED CATS

They demonstrated neurons in the striate cortex respond to stimuli in a **limited visual field,** which they called a **receptive field.**

They noted concentric overlapping responses, where complex patterns were combinations of lower-level patterns.

Their findings also revealed **specialization**, where some neurons would **only respond** to a **specific shape** or pattern.

In the 1980s, inspired by Hubel and Wielson, Kunihiko Fukushima published on the **neocognitron**, a neural network capable of learning patterns with geometrical similarity.

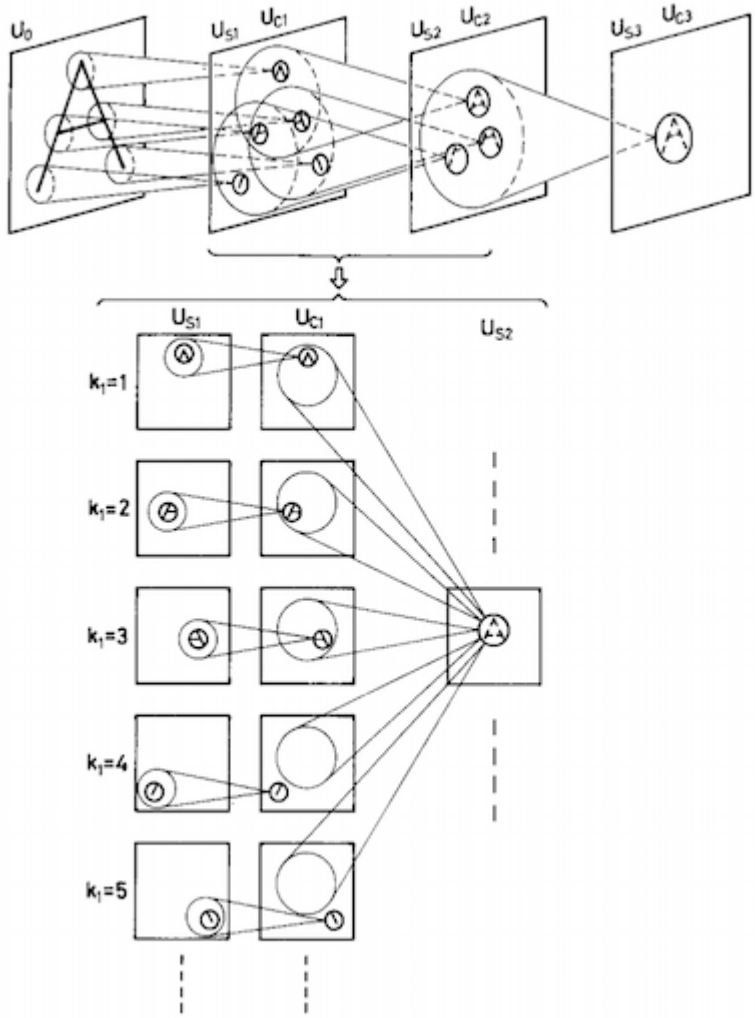


Fig. 5. An example of the interconnections between cells and the response of the cells after completion of self-organization

Neocognitron: A Self-organizing Neural Network Model for a Mechanism of Pattern Recognition Unaffected by Shift in Position

The **neocogitron** has two key properties:

- ***Learned patterns are hierarchal.*** Increasingly complex patterns are composed from simpler patterns.
- ***Learned patterns are position-invariant and translation-invariant.*** After the network learns a pattern, it can recognize the pattern at different locations. After **learning how to classify a dog**, the network can accurately classify an upside-down dog **without learning an entirely new pattern.**

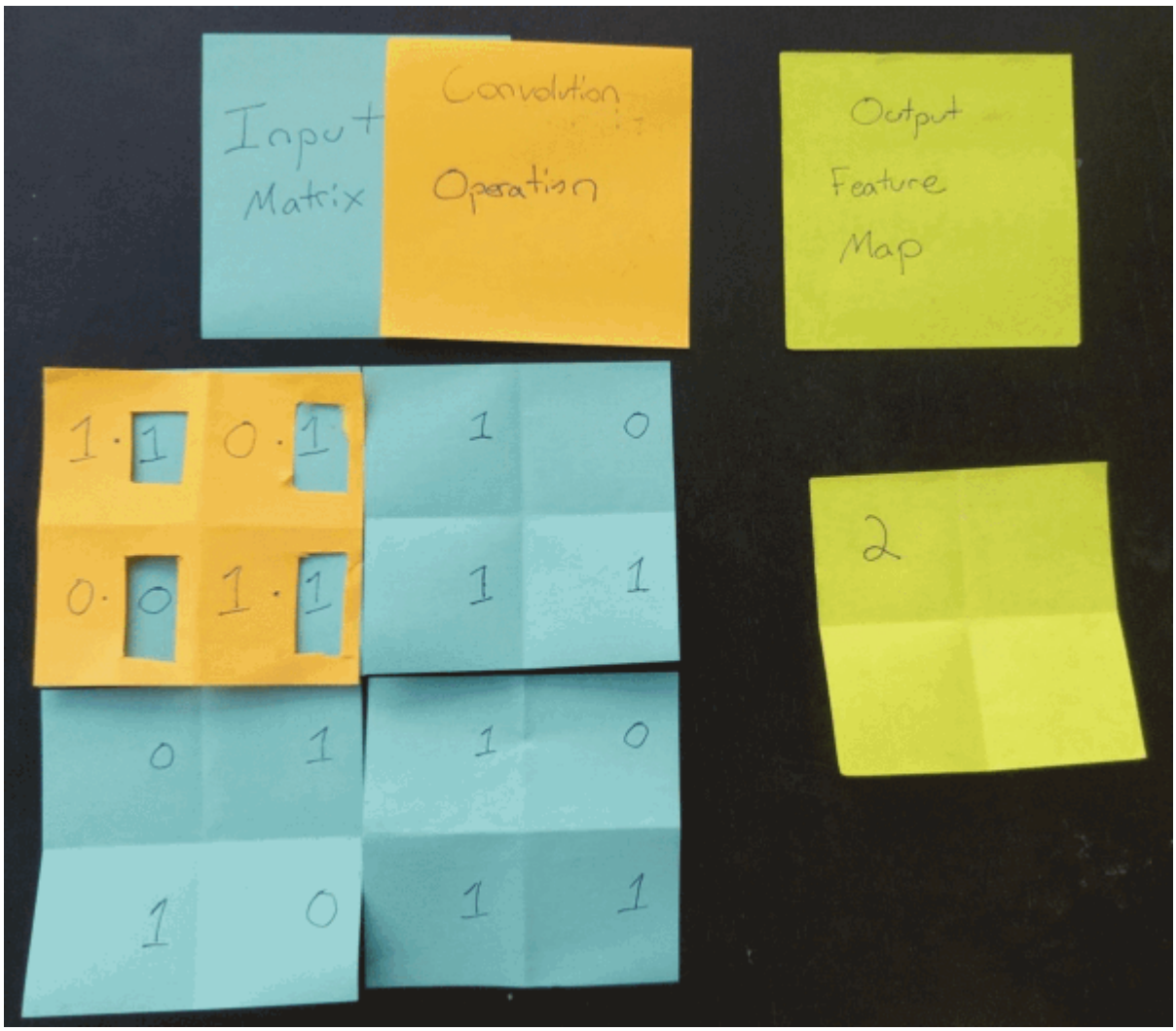
The **neocogitron** model is the inspiration for modern **convolutional neural networks.**

Visualizing a Convolution Operation: 2D

(Left) 2D 4×4 Input matrix. (Middle) 2D 2×2 kernel. (Right) 2D 2×2 output feature map.

The **input layer** is fed into **convolutional layers,** which transform **regions** of the input using a **filter.**

The **filter** is also referred to as a **kernel.**



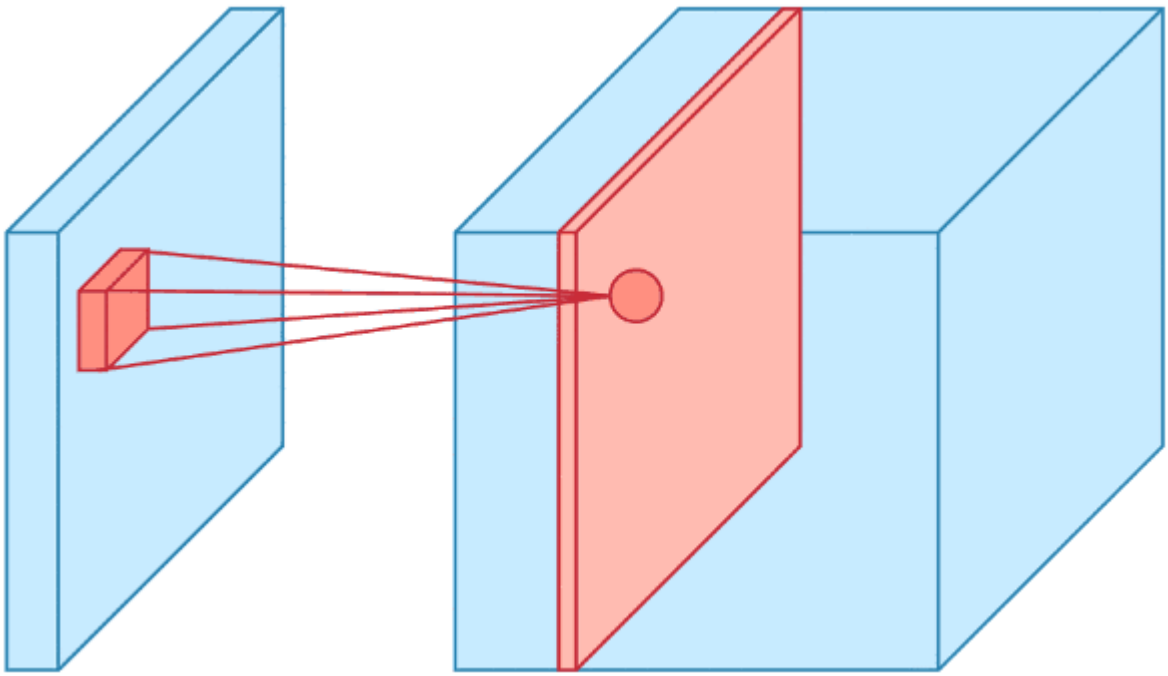
The filter "slides" to each possible position, and the result is added to the feature map.

For each position in the input matrix, the **convolution operation** performs matrix multiplication on each element.

The resulting matrix is summed and stored in a **feature map.**

The operation is repeated for each position in the input matrix.

Visualizing a Convolution Operation: 3D

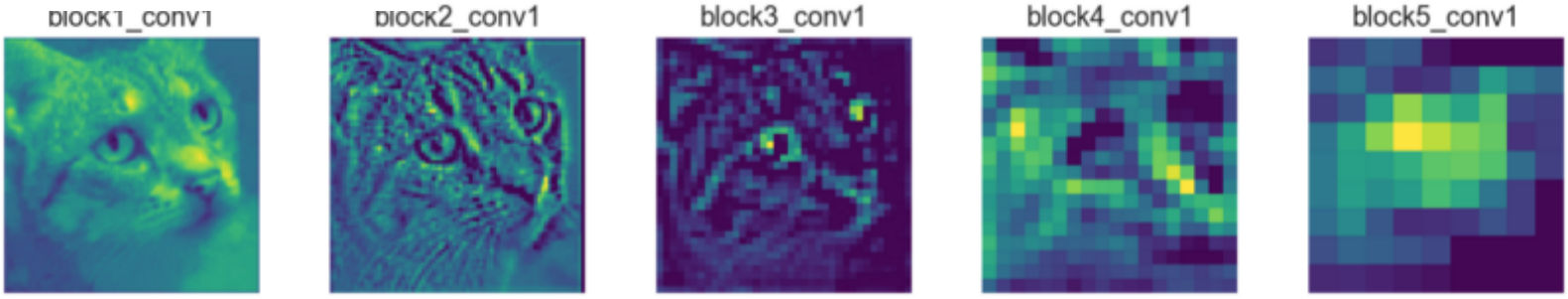


Applied Deep Learning—Part 4: Convolutional Neural Networks

The **input layer** of a CNN is usually a 3D data structure with ***height***, ***width***, and ***channel*** (RGB or greyscale values).

The deeper we go in the feature map stack, the **sparser** each map layer becomes. That means the filters detect fewer features.

The **first few layers** of the feature map stack **detect simple edges and shapes**, and look similar to the input image. As we go **deeper** into a feature map stack, **features become more abstract** to the human eye. Deeper feature layers **encode classification data,** like "cat face" or "cat ear".



Applied Deep Learning—Part 4: Convolutional Neural Networks

Do you want to learn more about CNNS?

Your dependency installation is probably done by now. To forge ahead, skip to **Part 8 - Deploy Pre-trained Model MobileNetV2.**

If you plan on training a custom classifier or want to read more about convolutional neural networks, start here:

- Applied Deep Learning—Part 4: Convolutional Neural Networks
- Hands on Machine Learning with Scikit-learn and TensorFlow, *Chapter 13, Convolutional Neural Networks*, by Aurélien Géron
- Deep Learning with Python, *Chapter 5 Deep Learning for Computer Vision,* by Francois Chollet

Part 8—Deploy Pre-trained Model (MobileNetV2)

Live Demo (using TensorFlow 2.0)

1. SSH into your Raspberry Pi

```
$ ssh raspberrypi.local
```

2. Start a new tmux session

```
pi@raspberryi:~ $ tmux new-session -s mobilenetv2
```

3. Split the tmux session vertically by pressing control+b, then “

4. Start an fbcp process, which will copy framebuffer from the PiCamera to the TFT display via SPI interface. Leave this process running.

```
pi@raspberryi:~ $ fbcp
```

5. Switch tmux panes by pressing control+b, then o.

6. Activate the virtual environment installed in earlier, in Part 6.

```
pi@raspberryi:~ $ cd ~/rpi-vision && . .venv/bin/activate
```

7. Start a mobilenetv2 agent process. The agent will take roughly 60 seconds to initialize.

```
pi@raspberryi:~/rpi-vision $ python rpi_vision/agent/mobilenet_v2.py
```

You’ll see a summary of the model’s base, and then the agent will print inferences until stopped. Click for a gist of what you should see.

This demo uses weights for **ImageNet** classifiers, which you can look up at image-net.org.

Wrapping Up

Congratulations, you just deployed an image classification model to your Raspberry Pi! ✨

Follow me @grepLeigh to get updates on this blog series. In my **next post**, I will show you how to:

1. Convert a model to **TensorFlow Lite,** a model format optimized for embedded and mobile devices.

2. Accelerate inferences of any **TensorFlow Lite** model with Coral's **USB Edge TPU Accelerator** and **Edge TPU Compiler.**
3. Employ **transfer learning** to re-train MobileNetV2 with a **custom image classifier.**

Computer Vision | Prototypes | Python | Raspberry Pi | Getting Started | TensorFlow | TensorFlow Lite

SHARE

Bitsy AI Labs Newsletter

Get the latest insights directly to your inbox!

Your email address

SUBSCRIBE

Featured posts

3 Key Ingredients for Embedded Computer Vision Apps
23 Jan 2023

PrintNanny Newsletter [01.2023]
06 Jan 2023

I needed a sold-out SDWire board, so I learned how to fab PCBs
16 Nov 2022

Soft-launching an AI/ML Product as a Solo Founder
18 Apr 2021

3 Ways to Install TensorFlow 2 on Raspberry Pi

21 Nov 2020

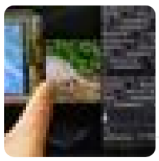
Latest posts

Open-Sourcing my Solo Founder Application to YC Winter 23 (PrintNanny.ai)

11 Sep 2022

zsh: no matches found

02 Nov 2020



Portable Computer Vision: Tensorflow 2 on a Raspberry Pi

23 Jun 2019

Start the conversation

Become a member of **Bitsy AI Labs** to start commenting.

Sign up now

Already a member? Sign in

You might also like

Computer Vision • 4 min read • 0

3 Key Ingredients for Embedded Computer Vision Apps

Leigh Johnson
23 Jan 2023

AutoML • 20 min read • 0

Soft-launching an AI/ML Product as a Solo Founder

Leigh Johnson
18 Apr 2021

Prototypes • 10 min read • 0

Automate Image Annotation on a Small Budget

Leigh Johnson
02 Nov 2020

Bitsy AI Labs

👋 Hi, my name is Leigh. I'm building PrintNanny, an automation assistant for 3D printers. I write about embedded AI/ML products and my journey from a software Tech Lead to a hardware Founder.

Navigation

Home
PrintNanny
RPI Pan-Tilt Tracking
Discord
Follow on Medium
Follow on Twitter

Subscribe

Your email address

JOIN NOW

©2024 Bitsy AI Labs. Published with Ghost & Nikko.